

Informe de evolución del RPA IndiGO a agente de IA supervisado

Fecha: 15 de julio de 2026

Estado: análisis y plan propuesto; implementación pendiente de aprobación.

1. Resumen ejecutivo

La evolución del RPA IndiGO a un agente de IA supervisado es técnicamente factible.

El RPA actual ya dispone de varias capacidades necesarias:

- Percepción de la interfaz mediante OpenCV, OCR y ventanas Win32.
- Clasificación de fallos operativos.
- Modo standby.
- Login y relogin automáticos.
- Reanudación desde la primera cédula pendiente.
- Registro de resultados por municipio.
- Notificaciones mediante Telegram.

Por tanto, no se recomienda reemplazar el RPA ni reescribirlo completamente. La evolución debe hacerse de forma incremental, separando progresivamente:

1. Percepción.
2. Estado.
3. Decisión.
4. Acción.
5. Verificación del resultado.
6. Retroalimentación.

La IA debe funcionar inicialmente como copiloto: analiza el estado y propone una acción estructurada. Un motor de políticas valida si la acción está permitida y determina si puede ejecutarse automáticamente o necesita aprobación humana.

No se debe permitir que una respuesta libre de ChatGPT se convierta directamente en clics, comandos de PowerShell, cierre de procesos o manipulación del escritorio.

2. Escenario operativo

El proceso se ejecuta actualmente en nueve equipos Windows. Cada equipo:

- Tiene una copia local del RPA.

- Trabaja sobre una carpeta o municipio diferente.
- Opera la interfaz local de IndiGO/Vie Cloud.
- Procesa una lista de cédulas.
- Consulta los folios de historias clínicas.
- Genera archivos PDF.
- Registra el último resultado en `resultados_indigo.csv`.

Los principales cuellos de botella reportados son:

- Lentitud de la interfaz WebView.
- Caídas o cierres de IndiGO.
- Cierre de sesión.
- Necesidad de reiniciar la aplicación.
- Reingreso de correo y contraseña.
- Esperas prolongadas durante la impresión.
- Falta de visibilidad central sobre los nueve equipos.
- Falta de informes automáticos consolidados.
- Intervención humana para reanudar ejecuciones.

3. Arquitectura actual del RPA

3.1 Tecnología

El RPA es una automatización de escritorio Windows escrita en Python.

Sus componentes principales son:

- `indigo_historias_core.py` : núcleo monolítico de aproximadamente 17.565 líneas.
- `run_indigo_historias.py` : ejecución normal con login.
- `reanudar_indigo_historias.py` : reanudación aprovechando una sesión abierta.
- `flujo_indigo.txt` : definición del flujo operativo por cédula.
- `indigo_produccion.env.bat` : configuración de producción.
- `metricas_indigo.py` : resumen local de resultados.

Tecnologías utilizadas:

- OpenCV y NumPy para reconocimiento visual.
- Tesseract OCR como respaldo.
- PyAutoGUI y PyWinAuto para teclado y ratón.
- Win32 y `ctypes` para ventanas y diálogos.
- MSS para capturas de pantalla.
- OpenPyXL para lectura de Excel.
- CSV y JSON para persistencia y configuración.

3.2 Flujo actual

El flujo general es:

1. Cargar la configuración.
2. Seleccionar la carpeta del municipio o E.S.E.
3. Leer las cédulas del archivo de entrada.
4. Consultar el registro histórico.
5. Omitir cédulas con estado final exitoso.
6. Abrir IndiGO.
7. Realizar login en Outlook/IndiGO.
8. Seleccionar la E.S.E.
9. Navegar hasta consulta de historias.
10. Procesar cada cédula.
11. Consultar folios.
12. Ejecutar la impresión.
13. Guardar el PDF.
14. Registrar el resultado.
15. Reintentar, entrar en standby o reloguear según el fallo.

3.3 Estados existentes

El RPA ya clasifica resultados como:

- OK
- SIN_HISTORIAS
- NO_ENCONTRADO
- INTERFAZ_CAIDA
- ESPERA_IMPRESION
- ERROR

Esta clasificación es una base valiosa para crear una máquina de estados explícita y transmitir eventos al dashboard.

4. Evidencia operativa disponible

En el workspace solo se encontró un archivo `resultados_indigo.csv` , correspondiente a:

008 – Barbosa

Resultados observados:

- Total de registros: **12.189**
- Estado OK : **3.797** — 31,15 %

- Estado SIN_HISTORIAS : **8.390** — 68,83 %
- Estado INTERFAZ_CAIDA : **1**
- Estado NO_ENCONTRADO : **1**
- Ventana temporal: 31 de mayo a 23 de junio de 2026.

Esta información representa un solo municipio/equipo y no permite extrapolar con rigor la disponibilidad ni el rendimiento de toda la flota.

Además, el CSV:

- Conserva principalmente el último estado por cédula.
- No representa el historial completo de incidentes y recuperaciones.
- Presenta desorden temporal.
- Contiene al menos un registro duplicado.
- Tiene identificadores con formatos no uniformes.

Por ello, la presencia de solo dos estados de fallo no significa que únicamente hayan ocurrido dos incidentes.

5. Cuellos de botella y riesgos

5.1 Caídas y sesiones

El RPA puede detectar una caída y ejecutar relogin, pero no existe:

- Heartbeat central.
- Vista de flota.
- Medición del tiempo de recuperación.
- Historial durable de cada intento.
- Escalamiento central cuando se agotan los reintentos.

5.2 Impresión y generación de PDF

La impresión es uno de los puntos más frágiles:

- Intervienen diálogos de IndiGO y Windows.
- Los PDF grandes pueden tardar varios minutos.
- El foco de las ventanas puede cambiar.
- El estado OK no verifica posteriormente el archivo.

Se debe validar:

- Existencia del PDF.
- Tamaño mínimo.
- Estabilidad del archivo.

- Integridad básica.
- Hash.
- Asociación con el municipio y paciente correctos.

5.3 Reanudación

La reanudación por CSV ya funciona, pero el archivo completo se reescribe después de cada cédula.

Riesgos:

- Corrupción ante un cierre durante la escritura.
- Ausencia de transacciones.
- Pérdida del historial de intentos.
- Dificultad para sincronizar información con el servidor.

Se recomienda SQLite como persistencia local transaccional y mantener el CSV únicamente como exportación compatible.

5.4 Operación en nueve equipos

Actualmente la asignación de municipios es principalmente manual.

No existe:

- Identidad segura por instalación.
- Lease de trabajo por municipio.
- Mutex local.
- Prevención de dos ejecuciones simultáneas.
- Inventario central de versiones y configuración.
- Watchdog externo.

5.5 Informes

`metricas_indigo.py` produce un resumen local en consola, pero no genera:

- Informe programado.
- Consolidado de nueve equipos.
- Historial de incidentes.
- Comparación entre municipios.
- Disponibilidad.
- Tiempo medio de recuperación.
- Reporte verificable de archivos descargados.

5.6 Seguridad

Se encontraron secretos operativos almacenados en texto plano. Deben considerarse comprometidos y rotarse antes de conectar los equipos con un servidor central.

También se detectaron rutas que permiten desactivar la validación TLS para Telegram.

Acciones obligatorias:

- Rotar credenciales y tokens expuestos.
- Retirar secretos del workspace.
- Usar Windows Credential Manager o DPAPI.
- Mantener TLS y validación de certificados habilitados.
- Definir política de retención de CSV, PDF, logs y capturas.
- Redactar cédulas en telemetría.
- No enviar capturas clínicas ni códigos de autenticación a la IA.

La ausencia de MFA facilita el login automatizado, pero aumenta el riesgo de apropiación de la cuenta. Se recomienda una cuenta exclusiva para automatización, con permisos mínimos, monitoreo y rotación.

6. Evaluación del proyecto web de referencia

El proyecto de referencia utiliza:

- Vue 3.
- Vue Router.
- Pinia.
- Express.
- Socket.IO.
- MongoDB/Mongoose.
- Separación lógica de bases por tenant.
- Autenticación mediante Google ID token.
- Inferencia mediante Azure GitHub Models.

Ya existen:

- `CompanyRpaDashboardView.vue`
- `p2lRpaDashboardApi.js`
- `p2lRpaDashboardRoutes.js`
- `p2lRpaDashboardService.js`
- `rpaDashboardModels.js`
- Tenant `p2l-rpa-dashboard`

Sin embargo, el dashboard RPA actual está orientado a planes, descargas y monitoreo SECOP. No administra el estado operativo de agentes locales.

6.1 Elementos reutilizables

- Autenticación Google.
- Validación de propiedad por empresa.
- Separación Mongo por tenant.
- CompanyDashHero .
- Cliente REST con Bearer token.
- Socket.IO.
- Historial de ejecuciones.
- Integración con inferencia GPT.
- Sistema de rutas white label.

6.2 Elementos que no deben copiarse literalmente

`PlgAnalyticsDashboardView.vue` es útil como referencia visual de eventos en tiempo real, pero su implementación actual:

- Usa una clave administrativa compartida.
- Combina polling permanente y Socket.IO.
- No está aislada por empresa.
- Presenta incompatibilidad con el middleware global del socket.

El canal RPA debe tener autenticación y rooms específicas por empresa y agente.

6.3 White label

El white label existente es principalmente estático:

- Rutas específicas.
- Assets específicos.
- CSS específico.
- Condicionales en el login.

Para esta evolución se recomienda incorporar una configuración de marca obtenida del backend:

- Nombre.
- Logo.
- Colores.
- Textos.
- Dominio.
- Funcionalidades habilitadas.
- Política de autonomía.

7. Arquitectura objetivo

Dashboard Vue

- |
- |— REST: consultas, informes, creación de comandos y aprobaciones
- |— Socket.IO: estados, eventos, alertas y progreso
- |

Control plane Express

- |
- |— MongoDB: agentes, ejecuciones, eventos, comandos y auditoría
- |— Motor de políticas
- |— Servicio de inferencia IA
- |— Cola durable de comandos
- |
- | HTTPS/WSS saliente
- |

9 agentes RPA Windows

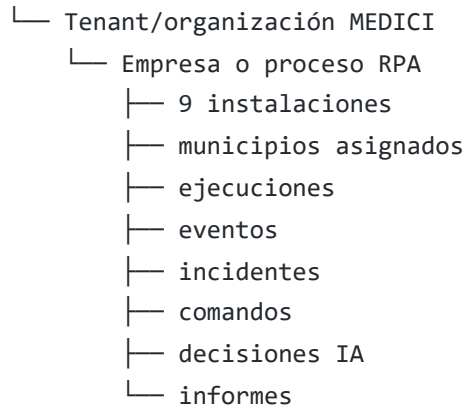
- |— Percepción OpenCV/OCR/Win32
- |— Máquina de estados
- |— Ejecutor de acciones permitidas
- |— SQLite y outbox
- |— Watchdog
- |— Verificación de PDF

Los agentes deben iniciar conexiones salientes hacia el servidor. No se recomienda que el backend intente acceder directamente a los equipos.

8. Modelo n-1-n recomendado

No es necesario crear nuevos tenants.

Plataforma



Entidades mínimas:

- RpaAgent
- MunicipalityAssignment
- RpaRun
- RpaEvent
- RpaIncident
- RpaCommand
- RpaCommandAttempt
- RpaDecision
- RpaArtifact
- RpaDailyReport

9. Máquina de estados propuesta

Estados del agente:

- OFFLINE
- STARTING
- AUTHENTICATING
- READY
- RUNNING
- WAITING_PRINT
- STANDBY
- RECOVERING
- PAUSED
- COMPLETED
- STOPPING

- ERROR

Cada transición debe generar un evento con:

- event_id
- run_id
- agent_id
- company_id
- municipality_code
- Timestamp UTC.
- Estado anterior.
- Estado nuevo.
- Severidad.
- Motivo.
- Intento.
- Referencia de paciente protegida.

Las cédulas no deben transmitirse en claro. Puede utilizarse un HMAC estable para correlacionar eventos sin revelar el identificador.

10. Eventos mínimos

- agent.started
- agent.ready
- agent.heartbeat
- agent.paused
- agent.resumed
- agent.stopped
- login.started
- login.completed
- login.failed
- session.reused
- patient.started
- patient.completed
- workflow.step.started
- workflow.step.completed
- workflow.step.failed
- print.started
- print.completed
- print.timed_out

- `pdf.saved`
- `pdf.verification_failed`
- `recovery.started`
- `recovery.attempted`
- `recovery.completed`
- `recovery.exhausted`
- `command.received`
- `command.accepted`
- `command.rejected`
- `command.completed`

11. Canal de comandos

Comandos permitidos inicialmente:

- `status.get`
- `ui.reobserve`
- `run.pause`
- `run.resume`
- `run.stop_graceful`
- `patient.retry_current`
- `indigo.relogin`
- `diagnostics.capture`
- `config.reload_safe`

No se deben permitir:

- Shell remoto arbitrario.
- PowerShell remoto genérico.
- Ejecución de binarios enviados por el servidor.
- Coordenadas de clic suministradas libremente.
- `taskkill` arbitrario.
- Cambio remoto de credenciales.

Cada comando debe incluir:

- UUID.
- Agente de destino.
- Empresa.
- Usuario solicitante.
- Acción.
- Versión del payload.

- Clave de idempotencia.
- Fecha de creación.
- Fecha de expiración.
- Estado.
- Intento.
- Resultado.

Flujo:

```
queued → delivered → accepted → running
                                └─ succeeded
                                └─ failed
                                └─ rejected
                                └─ expired
```

12. Uso de inferencia IA

El patrón existente de `CompanyJobCopilotDashboardView.vue` y Azure GitHub Models puede reutilizarse creando un servicio específico, por ejemplo:

`rpaDecisionService`

Entrada:

- Estado actual.
- Últimos eventos.
- Capturas previamente redactadas cuando sean indispensables.
- Número de intentos.
- Estado de IndiGO.
- Estado de impresión.
- Último comando.
- Política de autonomía.

Respuesta estructurada:

```
{
  "classification": "INDIGO_SESSION_EXPIRED",
  "confidence": 0.93,
  "recommended_action": "indigo.relogin",
  "reason": "La ventana de consulta no está disponible y se detectó la pantalla de a",
  "requires_approval": true,
  "preconditions": [
    "agent.state == STANDBY",
    "active_command == null"
  ]
}
```



El modelo no ejecuta la acción. El motor de políticas:

1. Valida el esquema.
2. Comprueba la allowlist.
3. Evalúa precondiciones.
4. Revisa el nivel de riesgo.
5. Solicita aprobación cuando aplique.
6. Crea un comando durable.
7. Espera ACK.
8. Verifica el nuevo estado.
9. Registra el resultado como retroalimentación.

Niveles de autonomía

Nivel 0 — Observación

- Diagnóstico.
- Resumen.
- Informe.
- Recomendación.

Nivel 1 — Acciones automáticas de bajo riesgo

- Reobservar interfaz.
- Refrescar estado.
- Reintentar transporte de telemetría.
- Recargar configuración segura.

Nivel 2 — Aprobación humana

- Reanudar ejecución.

- Reintentar paciente.
- Reloqear IndiGO.
- Detener ordenadamente.

No se recomienda habilitar autonomía superior durante el piloto.

13. Dashboard de control propuesto

13.1 Vista de flota

- Nueve equipos.
- Estado actual.
- Último heartbeat.
- Municipio asignado.
- Versión del agente.
- Ejecución activa.
- Tiempo en el estado actual.

13.2 Ejecución en tiempo real

- Total de cédulas.
- Procesadas.
- Pendientes.
- PDF verificados.
- Sin historias.
- Fallos.
- Cédula protegida actual.
- Paso actual.
- Tiempo por paso.

13.3 Incidentes

- Caídas de IndiGO.
- Sesiones expiradas.
- Esperas de impresión.
- Fallos de plantilla.
- Reintentos.
- Tiempo de recuperación.
- Incidentes pendientes de aprobación.

13.4 Copiloto IA

- Contexto del incidente.
- Diagnóstico.
- Confianza.
- Acción recomendada.
- Precondiciones.
- Botón aprobar/rechazar.
- Resultado observado.

13.5 Informes

Informe automático del proceso:

- Disponibilidad por equipo.
- Horas activas.
- Tiempo en standby.
- Caídas.
- Recuperaciones.
- Reintentos.
- Avance por municipio.

Informe automático de descargas:

- PDF verificados.
- Archivos fallidos.
- Tamaño total.
- Hash.
- Duplicados.
- Archivos faltantes.
- Duración media.
- Distribución por municipio.

Los informes deben poder programarse diariamente y descargarse en CSV/PDF.

14. Plan de trabajo por fases

Fase 0 — Seguridad y línea base

Objetivos:

- Rotar secretos.
- Retirar credenciales del workspace.
- Habilitar TLS obligatorio.

- Inventariar los nueve equipos.
- Confirmar municipios asignados.
- Definir roles, SLA y tratamiento de datos.

Puerta de salida:

- Autorización formal para conectar un equipo piloto.

Fase 1 — Instrumentación local

Objetivos:

- Incorporar máquina de estados.
- Generar heartbeat.
- Crear eventos estructurados.
- Añadir `run_id` y `agent_id` .
- Implementar SQLite y outbox.
- Añadir mutex de instancia.
- Verificar los PDF.

Puerta de salida:

- Un equipo observable localmente sin afectar el flujo productivo.

Fase 2 — Control plane n-1-n

Objetivos:

- Ampliar el tenant `p2l-rpa-dashboard` .
- Crear modelos Mongo.
- Implementar API de agentes y eventos.
- Crear identidad individual por instalación.
- Implementar Socket.IO autenticado.
- Persistir comandos, ACK y leases.

Puerta de salida:

- Telemetría durable de un equipo en ambiente piloto.

Fase 3 — Dashboard white label

Objetivos:

- Evolucionar `CompanyRpaDashboardView.vue` .
- Incorporar vista de flota.
- Mostrar eventos y estados en tiempo real.
- Mostrar avance y descargas.

- Incorporar incidentes y alertas.
- Crear informes automáticos.

Puerta de salida:

- Operador puede entender el estado del proceso sin acceder al equipo.

Fase 4 — Comandos supervisados

Objetivos:

- Pausar.
- Reanudar.
- Reobservar.
- Reintentar paciente.
- Relogear IndiGO.
- Detener ordenadamente.
- Registrar usuario, motivo y resultado.

Puerta de salida:

- Todos los comandos son idempotentes, auditados y verificados.

Fase 5 — Copiloto IA

Objetivos:

- Crear servicio de decisiones RPA.
- Generar recomendaciones JSON.
- Aplicar política de acciones.
- Incorporar aprobación humana.
- Registrar confianza, decisión y resultado.

Puerta de salida:

- La IA diagnostica incidentes sin ejecutar acciones fuera de la allowlist.

Fase 6 — Autonomía gradual y despliegue

Objetivos:

- Habilitar autoacciones de bajo riesgo.
- Establecer límites y rollback.
- Medir falsos positivos.
- Ejecutar piloto en un equipo.
- Ampliar a tres equipos.
- Desplegar finalmente en nueve.

Puerta de salida:

- Cumplimiento del SLA y ausencia de regresiones operativas o de seguridad.

15. Criterios mínimos de éxito

- Los nueve equipos reportan heartbeat.
- Toda ejecución tiene `run_id`.
- Todo incidente queda registrado.
- Los comandos tienen ACK y resultado.
- No existen secretos en texto plano dentro del código.
- Los PDF marcados como OK han sido verificados.
- No pueden ejecutarse dos instancias sobre el mismo municipio.
- El dashboard respeta aislamiento por empresa.
- La IA no ejecuta texto libre.
- Las acciones de riesgo requieren aprobación.
- Los informes de estado y descargas se generan automáticamente.
- La pérdida temporal de Internet no detiene el RPA ni pierde eventos.

16. Recomendación final

Se recomienda aprobar inicialmente solo la Fase 0.

No deben habilitarse telemetría central, comandos remotos ni inferencia sobre información operativa hasta:

1. Rotar los secretos expuestos.
2. Asegurar TLS.
3. Definir el tratamiento de cédulas, capturas, PDF y logs.
4. Confirmar roles y acciones autorizadas.
5. Seleccionar un único equipo piloto.

La arquitectura propuesta permite aprovechar el RPA existente, reducir la intervención manual y obtener control centralizado sin entregar a la IA control irrestricto sobre los equipos.